



Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1

Razvoj storitev na osnovi GraphQL

2

GraphQL

- Razvili so ga pri podjetju Facebook in ponudili kot odprtokodno rešitev
 - Najprej se je uporabljal v domorodnih aplikacijah v letu 2012
 - Javnosti se je prvič predstavil 2015 na konferenci React.js Conf
 - Kmalu po tem so objavili tudi novice o tem, da bo postal odprtokodni projekt
 - Za razvoj skrbi velika skupnost

3

GraphQL

- **Osnovna ideja:** pridobivanje podatkov na osnovi deklaracije poizvedbe v času izvajanja
- Odjemalec je tisti, ki specificira katere podatke potrebuje od APIja
- Strežnik je odgovoren, da odjemalcu zagotovi zahtevane podatke

4

GraphQL

- **API GraphQL = ena končna točka, ki zagotovi odjemalcem tiste podatke, ki jih zahtevajo**
- V ostalih tehnologijah običajno definiramo več končnih točk z vnaprej določenimi vmesniki
 - fiksni strukturi podatkov, ki jih odjemalci lahko pridobijo
 - tesna sklopljenost med storitvijo in obstoječimi odjemalci

5

GraphQL – povpraševalni jezik za APIje

- Tehnologijo GraphQL pogosto zamenjujejo s tehnologijo za upravljanje s podatki
- **GraphQL je povpraševalni jezik za APIje in ne podatkovne baze oz. podatke!**
- Je PB-agnostičen in se lahko učinkovito uporablja v kateremkoli kontekstu oz. v kombinaciji s katerimkoli mehanizmom za upravljanje s podatki

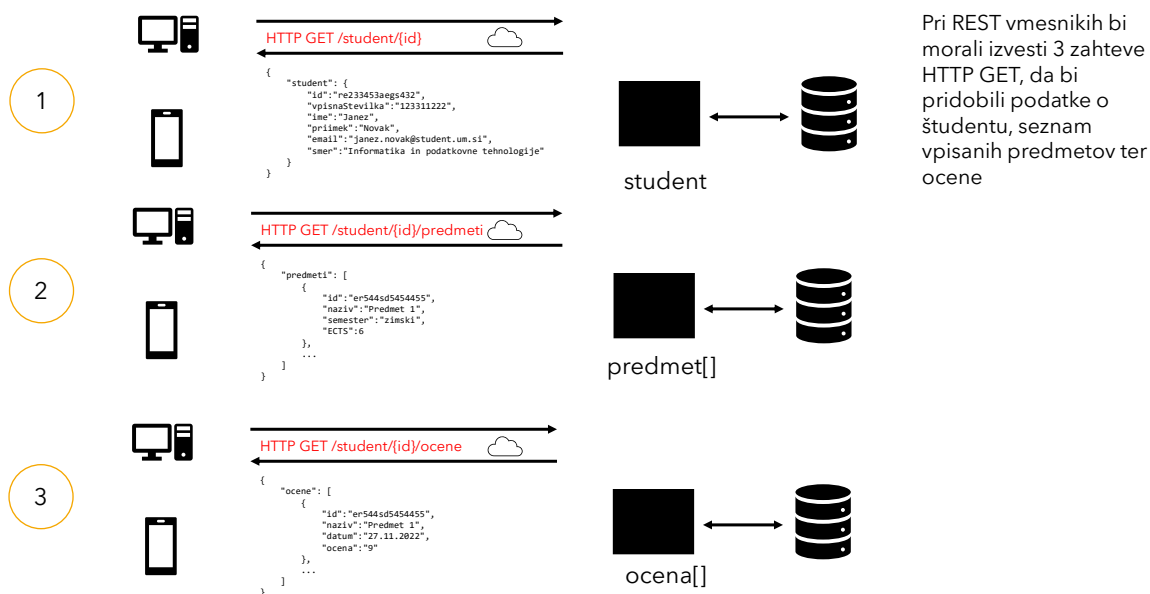
6

Pridobivanje podatkov na osnovi REST

- Recimo, da imamo storitev REST, ki omogoča pridobivanje podatkov o:
 - **študentu** - storitev ponuja podatke na **/student/{id}**
 - **predmetih** - storitev ponuja podatke na **/student/{id}/predmeti**
 - **ocenah** - podatki so na voljo na **/student/{id}/ocene**
- Za pridobitev vseh podatkov o študentu, o predmetih, ki jih opravlja v trenutnem letniku ter o ocenah, ki jih je pridobil, je potrebno poslati 3 ločene zahteve HTTP na strežnik

7

Pridobivanje podatkov z REST



8

Pridobivanje podatkov na osnovi GraphQL

- Največja prednost tehnologije GraphQL je v izrazni moči povpraševalnega jezika
- Omogoča definicijo poizvedb **query**
 - v kateri vključimo zahtevo po vseh podatkih, ki jih potrebujemo → strežnik je odgovoren, da zagotovi vse podatke skladno z povpraševanjem

9

Pridobivanje podatkov z GraphQL

```

query {
  student("id":"re233453aegs432") {
    vpisnaStevilka
    ime
    priimek
    email
    smer
    predmeti {
      naziv
      semester
      ECTS
    }
    ocene {
      naziv
      datum
      ocena
    }
  }
}

```

Z uporabo GraphQL **odjemalec specificira katere podatke potrebuje v zahtevi**, ki jo pošlje na strežnik. Odgovor strežnika je skladen s strukturo, ki je določena v zahtevi.



HTTP POST



```

{
  "data": {
    "student": {
      "id": "re233453aegs432",
      "vpisnaStevilka": "123311222",
      "ime": "Janez",
      "priimek": "Novak",
      "email": "janez.novak@student.um.si",
      "smer": "Informatika in podatkovne tehnologije",
      "predmeti": [
        {
          "id": "er544sd5454455",
          "naziv": "Predmet 1",
          "semester": "zimski",
          "ECTS": 6
        },
        ...
      ],
      "ocene": [
        {
          "id": "er544sd5454455",
          "naziv": "Predmet 1",
          "datum": "27.11.2022",
          "ocena": "5"
        },
        ...
      ]
    }
  }
}

```

10

GraphQL Schema za definicijo vmesnika storitve

- Definira zmožnosti APIja,
 - specificira kako lahko odjemalec pridobi podatke ali spremeni/posodobi obstoječe podatke
- **Schema je pogodba med odjemalcem in strežnikom**
- Schema je v bistvu zbirka GraphQL tipov definiranih s specialnimi korenskimi tipi (Query, Mutation in Subscription)
- Ima podobno vlogo kot
 - WSDL pri spletnih storitvah SOAP,
 - proto pri gRPC,
 - itd.

11

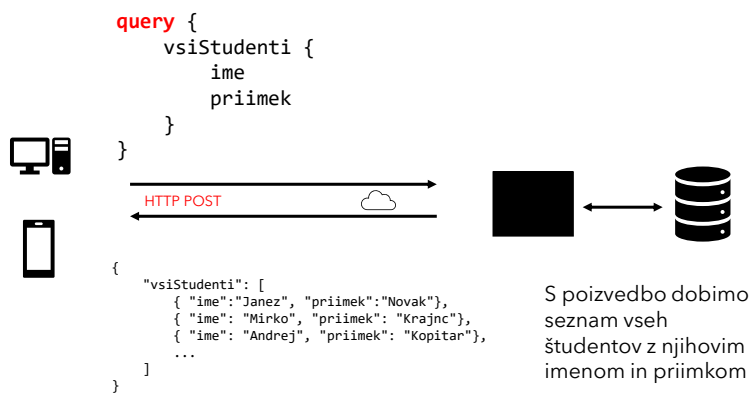
GraphQL – bralne in pisalne operacije

- **V splošnem je shema GraphQL zbirka GraphQL tipov**
- Posebnost jezika GraphQL je definicija bralnih in pisalnih operacij, ki jih zagotavlja vmesnik
- Definiramo lahko 3 tipe operacij
 - Type **Query** { ... }
 - Type **Mutation** { ... }
 - Type **Subscription** { ... }

} Vstopna točka za zahteve iz strani odjemalcev

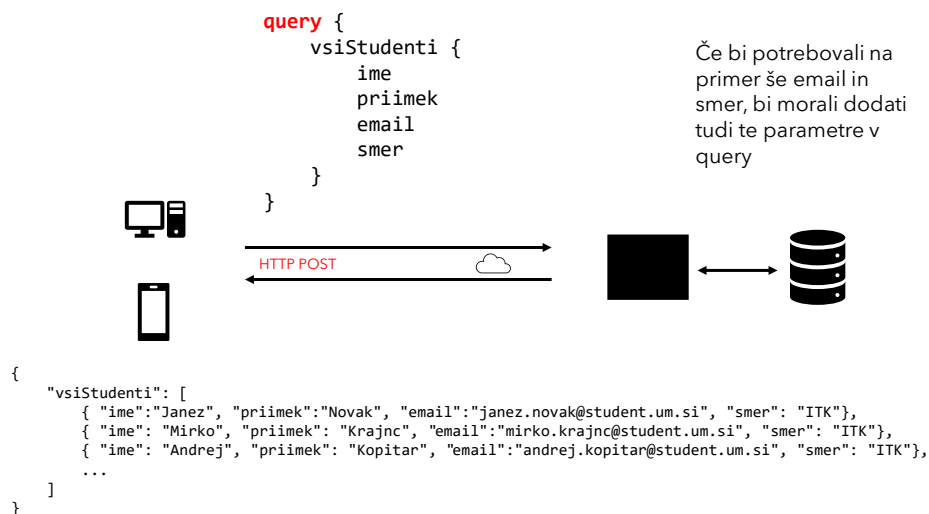
12

Pridobivanje podatkov z GraphQL



13

Pridobivanje podatkov z GraphQL



14

Gnezdenje poizvedb

- Ena izmed največjih izraznih moči jezika GraphQL je naravno gnezdenje informacij, ki jih potrebujemo iz strani strežnika.
- Če bi recimo želeli pridobiti vse ocene vseh študentov, sledimo strukturi tipov, ki so definirani v vmesniku, na primer:

```

query{
  vsiStudenti {
    ime
    priimek
    ocene {
      naziv
      datum
      ocena
    }
  }
}

```

```

{
  "vsiStudenti": [
    {
      "ime": "Janez",
      "priimek": "Novak",
      "ocene": [
        { "naziv": "Predmet1", "datum": "11.11.2022", "ocena": "10" },
        { "naziv": "Predmet2", "datum": "16.11.2022", "ocena": "9" }
      ]
    },
    {
      "ime": "Mirko",
      "priimek": "Krajnc",
      "ocene": [
        { "naziv": "Predmet2", "datum": "10.1.2022", "ocena": "8" },
        { "naziv": "Predmet3", "datum": "26.11.2022", "ocena": "7" }
      ]
    },
    ...
  ]
}

```

Strežnik bo razčlenil poizvedbo ter pridobil ocene ter vključil seznam ocen, ki so povezane s posameznim študentom

Primer rezultata poizvedbe

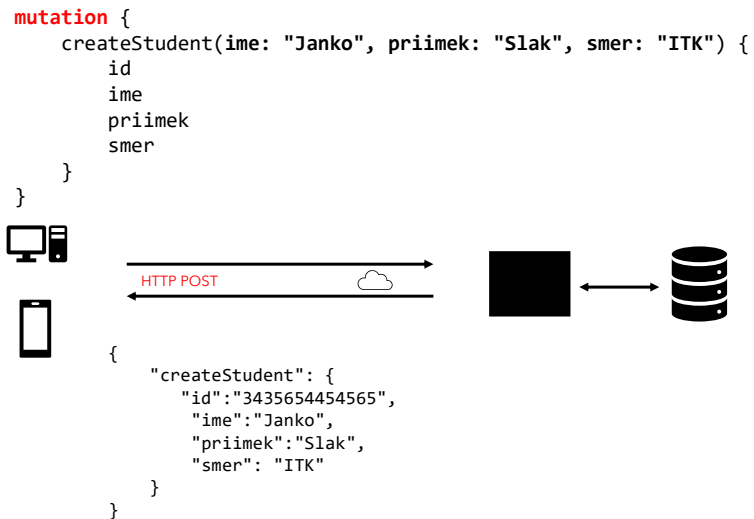
15

Pisanje podatkov

- Aplikacije poleg operacij za branje podatkov pogosto potrebujejo tudi operacije za spreminjanje vrednosti podatkov na strežniku
- V GraphQL se za operacije, ki omogočajo spreminjanje vrednosti na strežniku, definirajo mutacije, pri čemer v splošnem ločimo 3 oblike mutacij
 - **Ustvarjanje novih podatkov**
 - **Posodobitev obstoječih podatkov**
 - **Brisanje obstoječih podatkov**
- Sintaksa za definicijo mutacij je enaka kot pri definiciji bralnih operacij

16

Pisanje podatkov z mutacijami



17

Primer GraphQL APIja na osnovi programskega jezika Java in ogrodja Spring

18

GraphQL for Java

- **GraphQL Java** (<https://www.graphql-java.com/>) = Javanska implementacija GraphQL
 - Na Github je več repozitorijev, povezanih z GraphQL, od katerih je najpomembnejši GraphQL Java Engine - osnova za implementacijo strežnika.
- **GraphQL Java Engine** je ogrodje za izvajanje poizvedb GraphQL
 - Ne zagotavlja procesiranja zahtev HTTP, JSON
 - Za implementacijo procesiranja zahtev uporabimo na primer ogrodje **Spring for GraphQL**, ki zagotovi, da se API na osnovi ogrodja Spring izpostavi čez HTTP

19

Inicializacija projekta

```
mkdir student-gql-jpa && cd student-gql-jpa
```

```
curl -L "https://start.spring.io/starter.zip?type=maven-project&language=java&bootVersion=4.0.1&groupId=si.um&artifactId=student-gql-jpa&name=student-gql-jpa&packageName=si.um.studentgql&dependencies=web,graphql,websocket,data-jpa,h2,devtools" -o app.zip
```

```
unzip app.zip
```

```
rm app.zip
```

```
code .
```

20

Nastavitve: GraphQL + H2 in-memory + JPA

```
spring.application.name=student-gql-jpa

# --- GraphQL ---
spring.graphql.graphiql.enabled=true

# --- H2 in-memory + JPA/Hibernate ---
spring.datasource.url=jdbc:h2:mem:stddb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# H2 konzola
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

21

Nastavitve

```
spring.graphql.graphiql.enabled=true
spring.graphql.schema.printer.enabled=true
spring.graphql.websocket.path=/graphql
```

- GraphQL bo dostopen na **/graphiql**
- shema bo dostopna na **/graphql/schema**
- WebSocket za subscriptions je "off" privzeto;
 - vključi se z spring.graphql.websocket.path in (pri servlet stacku)

22

Dodamo GraphQL schema

- Spring Boot privzeto išče sheme v classpath:graphql/**, kar je tipično src/main/resources/graphql/.

- Ustvarimo datoteko:

src/main/resources/graphql/schema.graphqls

23

src/main/resources/graphql/schema.graphqls

```

type Query {
  getStudent(id: ID!): Student
  vsiPredmeti(id: ID!): [Predmet!]!
  vseOcene(id: ID!): [Ocena!]!
}

type Mutation {
  createStudent(ime:String!, priimek:String!, smer: String!): Student!
  updateStudent(id:ID!, ime:String!, priimek:String!, smer: String!, email:String!): Student!
  deleteStudent(id:ID!): Student!
  createPredmet(naziv:String!, semester:String!, ects: Int):Predmet!
  updatePredmet(id:ID!, naziv:String!, semester:String!, ects: Float):Predmet!
  deletePredmet(id:ID!):Predmet!
  createOcena(naziv:String!, ocena:Float!, datum: String!, studentId:ID!):Ocena!
  updateOcena(id:ID!, naziv:String!, ocena: Float!, datum: String!, studentId: ID!):Ocena!
  deleteOcena(id:ID!):Ocena!
}

type Student {
  id: ID!
  vpisnaStevilka: String!
  ime: String!
  priimek: String!
  email: String!
  smer: String!
  predmeti: [Predmet!]!
  ocene: [Ocena]
}

type Predmet {
  id: ID!
  naziv: String!
  semester: String!
  ects: Int!
}

type Ocena {
  id: ID!
  naziv: String!
  ocena: Float!
  datum: String!
  student: Student!
}

```

24

Dodamo entitete

- V paketu **si.um.studentgql.entity** dodamo naslednje entitete:
 - **StudentEntity**
 - **PredmetEntity**
 - **OcenaEntity**

25

StudentEntity

```
package si.um.studentgql.entity;
import jakarta.persistence.*;
@Entity
@Table(name = "students")
public class StudentEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String vpisnaStevilka;
    @Column(nullable = false)
    private String ime;
    @Column(nullable = false)
    private String priimek;
    @Column(nullable = false)
    private String email;
    @Column(nullable = false)
    private String smer;

    public StudentEntity() { }

    public StudentEntity(String vpisnaStevilka, String ime, String priimek, String email, String smer) {
        this.vpisnaStevilka = vpisnaStevilka;
        this.ime = ime;
        this.priimek = priimek;
        this.email = email;
        this.smer = smer;
    }

    + GETTER in SETTER METODE
}
```

26

PredmetEntity

```
package si.um.studentgql.entity;
import jakarta.persistence.*;

@Entity
@Table(name = "predmeti")
public class PredmetEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String naziv;
    @Column(nullable = false)
    private String semester;
    @Column(nullable = false)
    private Integer ects;

    public PredmetEntity() { }

    public PredmetEntity(String naziv, String semester, Integer ects) {
        this.naziv = naziv;
        this.semester = semester;
        this.ects = ects == null ? 0 : ects;
    }

    + GETTER in SETTER metode
}
```

27

OcenaEntity

```
package si.um.studentgql.entity;
import jakarta.persistence.*;

@Entity
@Table(name = "ocene")
public class OcenaEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String naziv;
    @Column(nullable = false)
    private Float ocena;
    @Column(nullable = false)
    private String datum;
    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id", nullable = false)
    private StudentEntity student;
    public OcenaEntity() { }
    public OcenaEntity(String naziv, Float ocena, String datum, StudentEntity student) {
        this.naziv = naziv;
        this.ocena = ocena;
        this.datum = datum;
        this.student = student;
    }
    + GETTER in SETTER metode
}
```

28

Repozitoriji (Spring Data JPA)

- V paketu **si.um.studentgql.repo** dodamo naslednje entitete:
 - **StudentRepository**
 - **PredmetRepository**
 - **OcenaRepository**

29

StudentRepository

```
package si.um.studentgql.repo;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import si.um.studentgql.entity.StudentEntity;  
  
public interface StudentRepository extends JpaRepository<StudentEntity, Long> { }
```

30

PredmetRepository

```
package si.um.studentgql.repo;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import si.um.studentgql.entity.PredmetEntity;  
  
public interface PredmetRepository extends JpaRepository<PredmetEntity, Long> { }
```

31

OcenaEntity

```
package si.um.studentgql.repo;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
import si.um.studentgql.entity.OcenaEntity;  
  
import java.util.List;  
  
public interface OcenaRepository extends JpaRepository<OcenaEntity, Long> {  
    List<OcenaEntity> findByStudent_Id(Long studentId);  
}
```

32

Implementacija GraphQL kontrolnika

- Spring za GraphQL omogoča programiranje kontrolnikov na osnovi anotacij
- Z anotacijo **@Controller** označimo razred, ki ga želimo izpostaviti v obli GraphQL APIja
- Uporabimo ga lahko za označevanje razreda, ki mu določimo vlogo komponente GraphQL, ki je sposobna presteči zahteve GraphQL

33

Implementacija GraphQL kontrolnika

- Druge oznake
 - **@SchemaMapping** - poveže metodo s poljem v GraphQL shemi
 - **@QueryMapping** - za označevanje metode, ki se doda med Query
 - **@MutationMapping** - za označevanje metode, ki se doda med Mutation
 - **@SubscriptionMapping** - za označevanje metode, ki se doda med Subscription
 - **@Argument**
- Več na: <https://docs.spring.io/spring-graphql/docs/current/reference/html/#controllers>

34

Dodajanje GraphQL kontrolerja

- V paketu **src/main/java/si/um/studentgql/graphql/** dodamo razred **StudentGraphqlController.java**
 - Query
 - Mutation
 - Subscription
 - mapiranje polj

35

StudentGraphqlController

```
package si.um.studentgql.graphql;

import org.springframework.graphql.data.method.annotation.*;
import org.springframework.stereotype.Controller;
import si.um.studentgql.entity.*;
import si.um.studentgql.repo.*;
import si.um.studentgql.subscriptions.Events;

import java.util.List;

@Controller
public class StudentGraphqlController {

    private final StudentRepository studentRepo;
    private final PredmetRepository predmetRepo;
    private final OcenaRepository ocenaRepo;

    public StudentGraphqlController(StudentRepository studentRepo,
        PredmetRepository predmetRepo,
        OcenaRepository ocenaRepo) {

        this.studentRepo = studentRepo;
        this.predmetRepo = predmetRepo;
        this.ocenaRepo = ocenaRepo;
    }

    + implementacija operacij Query in Mutation
}
```

36

Operacije **Query**

```
@QueryMapping
public StudentEntity getStudent(@Argument String id) {
    return studentRepo.findById(Long.parseLong(id)).orElse(null);
}

@QueryMapping
public List<PredmetEntity> vsiPredmeti(@Argument String id) {
    studentRepo.findById(Long.parseLong(id)).orElseThrow();
    return predmetRepo.findAll();
}

@QueryMapping
public List<OcenaEntity> vseOcene(@Argument String id) {
    return ocenaRepo.findByStudent_Id(Long.parseLong(id));
}
```

37

Operacije **Mutation** (Student)

```
@MutationMapping
public StudentEntity createStudent(@Argument String ime, @Argument String priimek, @Argument String smer) {
    String vprisna = "V" + System.currentTimeMillis(); // simple demo
    String email = (ime + "." + priimek).toLowerCase() + "@example.com";
    StudentEntity s = studentRepo.save(new StudentEntity(vprisna, ime, priimek, email, smer));
    return s;
}

@MutationMapping
public StudentEntity updateStudent(@Argument String id, @Argument String ime, @Argument String priimek,
    @Argument String smer, @Argument String email) {
    StudentEntity s = studentRepo.findById(Long.parseLong(id)).orElseThrow();
    s.setIme(ime);
    s.setPriimek(priimek);
    s.setSmer(smer);
    s.setEmail(email);
    s = studentRepo.save(s);
    return s;
}

@MutationMapping
public StudentEntity deleteStudent(@Argument String id) {
    StudentEntity s = studentRepo.findById(Long.parseLong(id)).orElseThrow();
    studentRepo.delete(s);
    return s;
}
```

38

Operacije **Mutation** (Predmet)

```
@MutationMapping
public PredmetEntity createPredmet(@Argument String naziv, @Argument String semester, @Argument Integer ects) {
    return predmetRepo.save(new PredmetEntity(naziv, semester, ects));
}

@MutationMapping
public PredmetEntity updatePredmet(@Argument String id, @Argument String naziv, @Argument String semester, @Argument Integer ects) {

    PredmetEntity p = predmetRepo.findById(Long.parseLong(id)).orElseThrow();
    p.setNaziv(naziv);
    p.setSemester(semester);
    if (ects != null) p.setEcts(ects);
    return predmetRepo.save(p);
}

@MutationMapping
public PredmetEntity deletePredmet(@Argument String id) {
    PredmetEntity p = predmetRepo.findById(Long.parseLong(id)).orElseThrow();
    predmetRepo.delete(p);
    return p;
}
```

39

Operacije Mutation (Ocena)

```
@MutationMapping
public OcenaEntity createOcena(@Argument String naziv, @Argument Float ocena, @Argument String datum, @Argument String studentId) {

    StudentEntity s = studentRepo.findById(Long.parseLong(studentId)).orElseThrow();
    OcenaEntity o = ocenaRepo.save(new OcenaEntity(naziv, ocena, datum, s));
    return o;
}

@MutationMapping
public OcenaEntity updateOcena(@Argument String id, @Argument String naziv, @Argument Float ocena, @Argument String datum,
@Argument String studentId) {

    StudentEntity s = studentRepo.findById(Long.parseLong(studentId)).orElseThrow();
    OcenaEntity o = ocenaRepo.findById(Long.parseLong(id)).orElseThrow();
    o.setNaziv(naziv);
    o.setOcena(ocena);
    o.setDatum(datum);
    o.setStudent(s);
    o = ocenaRepo.save(o);
    return o;
}

@MutationMapping
public OcenaEntity deleteOcena(@Argument String id) {
    OcenaEntity o = ocenaRepo.findById(Long.parseLong(id)).orElseThrow();
    ocenaRepo.delete(o);
    return o;
}
```

40

Zagon

./mvnw spring-boot:run

```

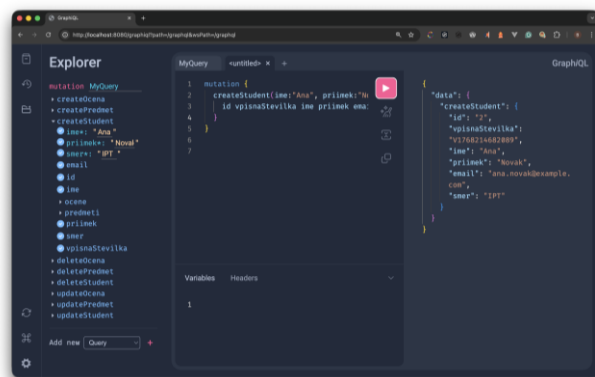
2026-01-12T11:38:55.025+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view
is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disa
ble this warning
2026-01-12T11:38:56.648+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] efaultSchemaResourceGraphQLSourceBuilder : Loaded 1 resource(s)
in the GraphQL schema.
2026-01-12T11:38:56.692+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] o.s.b.g.a.GraphQLAutoConfiguration : GraphQL schema inspect
ion:
  Unmapped fields: {Predmet={ECTS}}
  Unmapped registrations: {}
  Unmapped arguments: {}
  Field nullness errors: {}
  Argument nullness errors: {}
  Skipped types: {}
2026-01-12T11:38:56.698+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] s.b.g.a.s.GraphQLWebMvcAutoConfiguration : GraphQL endpoint HTTP
POST /graphql
2026-01-12T11:38:56.735+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] s.b.g.a.s.GraphQLWebMvcAutoConfiguration : GraphQL endpoint WebS
ocket /graphql
2026-01-12T11:38:56.739+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] o.s.b.h.h.H2ConsoleAutoConfiguration : H2 console available
at '/h2-console'. Database available at 'jdbc:h2:mem:studb'
2026-01-12T11:38:56.769+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on por
t 8080 (http) with context path '/'
2026-01-12T11:38:56.781+01:00 INFO 19510 --- [student-gql-jpa] [ restartedMain] s.u.studentgql.StudentGqlJpaApplication : Started StudentGqlJpa
Application in 1.245 seconds (process running for 1.379)

```

- GraphiQL: <http://localhost:8080/graphiql>
- H2 konzola: <http://localhost:8080/h2-console>
(JDBC URL: jdbc:h2:mem:studb)

41

Ustvarjanje študenta



```

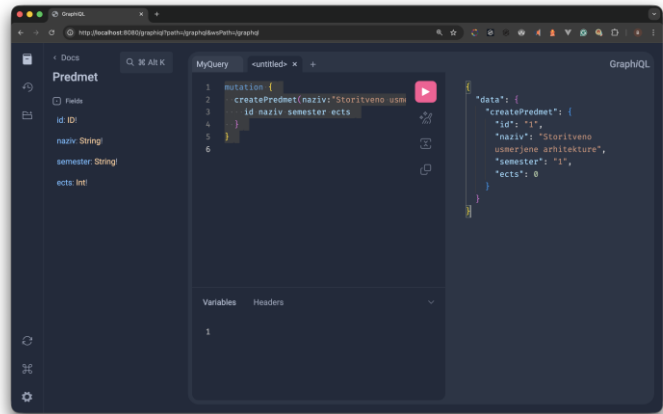
mutation {
  createStudent(ime:"Ana", priimek:"Novak",
    id vpisnaStevilka ime priimek email
    smer
  )
}

```

42

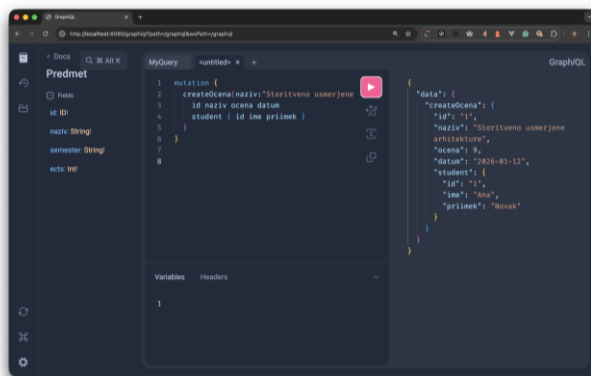
Ustvarjanje predmeta

```
mutation {
  createPredmet(naziv:"Storitveno
usmerjene arhitekture",
semester:"1",ects:6){
    id naziv semester ects
  }
}
```



43

Ustvarjanje ocene



```
mutation {
  createOcena(naziv:"Storitveno
usmerjene arhitekture",
ocena:9.0, datum:"2026-01-12",
studentid:"1") {
    id naziv ocena datum
    student { id ime priimek }
  }
}
```

44

Poizvedba

```
query {
  getStudent(id:"1"){
    id ime priimek email
    predmeti { id naziv ecta }
    ocene { id naziv ocena
    datum }
  }
}
```

